

Lab 1: Linux Basics, SSH, and IoT Sensor Integration

Author: Andrew J Napieraj

Class: ESCE 6660 Internetworking of Things (IoT)

Instructor: Prof. Ish Jain

TA: Chibuikem Ezemaduka

Due Date: February 2, 2026

Description

This lab evaluated basic Linux command-line usage, VirtualBox configuration, Secure Shell (SSH) connectivity, Raspberry Pi setup, and integration of basic IoT sensors (DHT11, RC522 RFID, and HC-SR04 ultrasonic ranger). The report is organized into Materials, Part 1 (VirtualBox, Linux Basics, and SSH), Part 2 (RPI and Sensors), and a Conclusion.

Part 1 shows the results from setting up the VirtualBox, logging into SSH for the Rensselaer Polytechnic Institute (RPI) homepage and posting the "bio.txt" to my personal RPI homepage. Basic Linux commands such as ls, cd, chmod, scp, and ssh were exercised to navigate the filesystem, manage permissions, and transfer files securely between systems. These commands were used extensively throughout the lab, particularly during file uploads to the RPI server and sensor script transfers to the Raspberry Pi.

Part 2 discusses the setup of the RPI to include:

- Sensor employment
- SPI Setup
- SSH Setup
- Initial Configuration

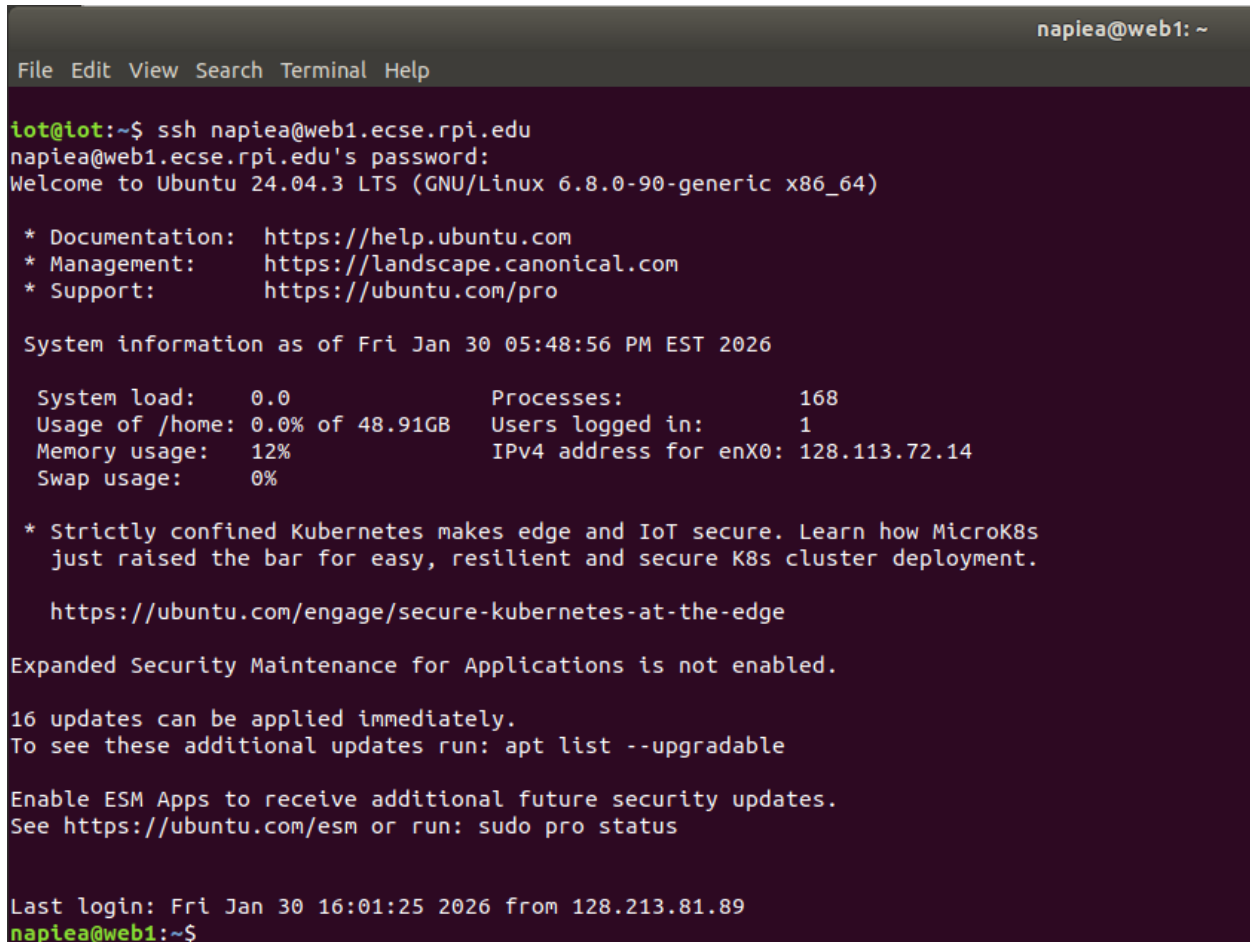
Materials

- 1 - RFID Reader (RC-522)
- 1 - Temperature–Humidity Sensor (DHT11)
- 1 - Ranger (HC-SR04)
- 1 - Wire connections (female-female connections)
- 1 - 1 kΩ Resistor (used as a voltage divider for HC-SR04 Echo pin level shifting)
- 1 - 2 kΩ Resistor (used as a voltage divider for HC-SR04 Echo pin level shifting)
- 1 - Raspberry Pi 3 Model B+
- 1 – 32 GB Micro SD Card
- 1 – Bluetooth Mouse/Keyboard (Optional)
- 1 – Full sized HDMI Cable (Optional)

Part 1: VirtualBox, Linux Basics and SSH

To start this section, we will discuss the documentation effectiveness for setting up VirtualBox. Overall, setup of VirtualBox was straightforward; once I realized the compressed file consisted of multiple parts, I used 7-Zip to extract the VM successfully. Because this was my first

exposure to Ubuntu, package manager lock states initially caused confusion during system updates; after consulting documentation and troubleshooting resources, I learned that these locks simply required waiting for background updates to complete. Once I was able to get past the initial wait for the daily updates, I was able to establish SSH on the VirtualBox as seen in *Figure 1*.

A screenshot of a VirtualBox terminal window. The title bar at the top right says 'napiea@web1: ~'. The menu bar includes 'File', 'Edit', 'View', 'Search', 'Terminal', and 'Help'. The terminal content shows a user 'iot@iot' running 'ssh napiea@web1.ecse.rpi.edu'. It prompts for a password, then shows the Ubuntu 24.04.3 LTS login banner. The banner includes links for documentation, management, and support. It also displays system information as of Fri Jan 30 05:48:56 PM EST 2026, including system load, processes, memory usage, and IPv4 address. A message about Kubernetes security is shown with a link. It states that expanded security maintenance for applications is not enabled and that 16 updates can be applied immediately. It provides instructions to see additional updates and to enable ESM apps. The last login is shown as Fri Jan 30 16:01:25 2026 from 128.213.81.89. The prompt returns to 'napiea@web1:~\$'.

```
napiea@web1: ~
File Edit View Search Terminal Help

iot@iot:~$ ssh napiea@web1.ecse.rpi.edu
napiea@web1.ecse.rpi.edu's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-90-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Jan 30 05:48:56 PM EST 2026

System load:  0.0               Processes:            168
Usage of /home: 0.0% of 48.91GB Users logged in:      1
Memory usage:   12%             IPv4 address for enX0: 128.113.72.14
Swap usage:     0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

16 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Fri Jan 30 16:01:25 2026 from 128.213.81.89
napiea@web1:~$
```

Figure 1: VirtualBox Terminal Showing SSH Enabled

After I verified proper functionality of the VM, I transferred to my localhost (closed the VM) to continue the rest of the project.

The next step after my homepage was created was to write a simple bio and upload it. A short biographical file was created and uploaded using scp as seen below in *Figure 2*, with the

attendant success message. The bio can be found at the following link:

<https://web1.ecse.rpi.edu/~napiea/bio.txt>

```
C:\Users\andre>scp "C:\Users\andre\Documents\bio.txt" napiea@web1.ecse.rpi.edu:~/public_html/
napiea@web1.ecse.rpi.edu's password:
bio.txt
100% 1372 30.5KB/s 00:00
C:\Users\andre>
```

Figure 2: Bio.txt Successful SCP Upload

After completing the file upload, I continued to test various commands, drawing some parallels to basic command prompt commands I have used previously.

Part 2: Raspberry Pi Setup and Sensor Employment

Setup of the Raspberry Pi (RPI) was straightforward utilizing the lab documents; I had a minor delay due to not having a microSD initially, but this was quickly remedied. Once I had the microSD, I set up the RPi using the Graphical User Interface (GUI) to set the appropriate network, and user settings. Once I completed the setup I connected to RPi using WinSCP to transfer python code over to run as seen in *Figure 3*. All scripts will be included in the Appendix for ease of reading.

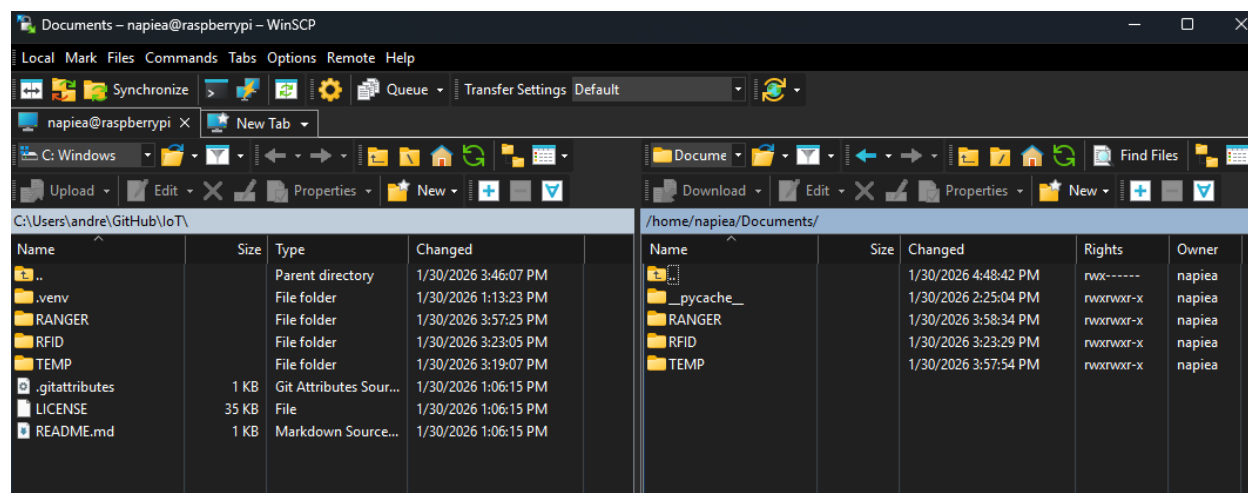
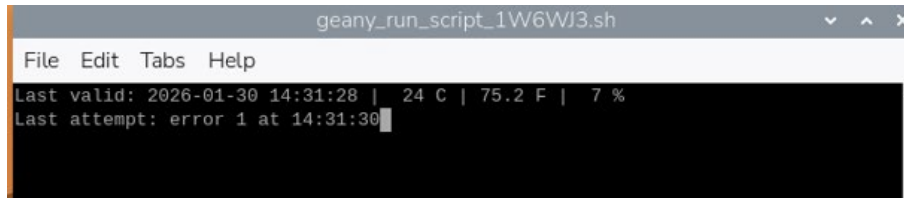


Figure 3: WinSCP RPi File Transfer

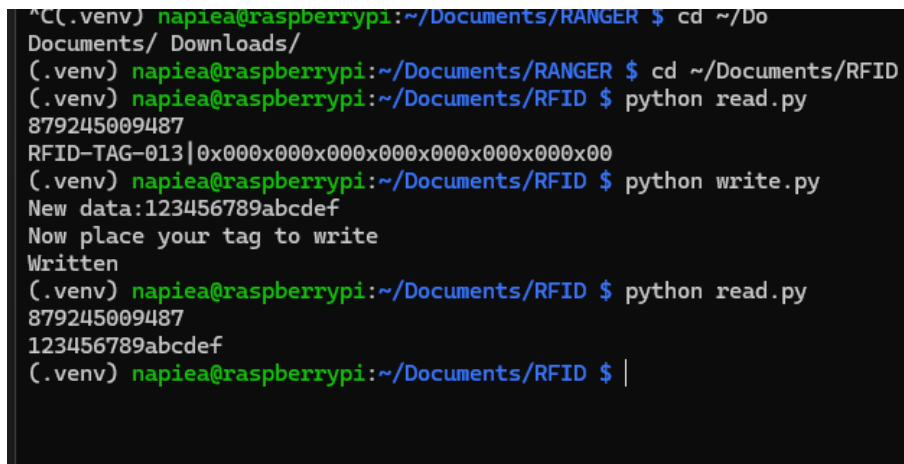
For the first sensor setup I simply used the HDMI connection to run the python script directly on the RPi for the temperature and humidity using the DHT11 as seen in *Figure 4*. I made some modifications to the base script provided^[1] allowing for a clean refresh every cycle and the ability to see the status of the sensor as it ran. What I noticed with this sensor is how sensitive the output of data is to the length of the wire. I also referenced the RPi pinout^[2] to get a better understanding of how the RPi is laid out.



```
geany_run_script_1W6WJ3.sh
File Edit Tabs Help
Last valid: 2026-01-30 14:31:28 | 24 C | 75.2 F | 7 %
Last attempt: error 1 at 14:31:30
```

Figure 4: DHT11 Temperature and Humidity Sensor Sample Output

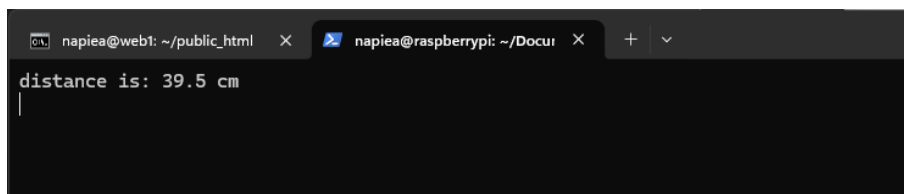
The next module I tested was the RFID Reader and Writer, or the RC522. This was also intuitive, and easy to use with the guide provided [3]. Below in *Figure 5* we can see a successful write operation and read operation post write for the RC522. The RC522 communicates with the Raspberry Pi over the SPI bus, which was enabled during system configuration.



```
^C(.venv) napiea@raspberrypi:~/Documents/RANGER $ cd ~/Do
Documents/ Downloads/
(.venv) napiea@raspberrypi:~/Documents/RANGER $ cd ~/Documents/RFID
(.venv) napiea@raspberrypi:~/Documents/RFID $ python read.py
879245009487
RFID-TAG-013|0x000x000x000x000x000x000x000x000x00
(.venv) napiea@raspberrypi:~/Documents/RFID $ python write.py
New data:123456789abcdef
Now place your tag to write
Written
(.venv) napiea@raspberrypi:~/Documents/RFID $ python read.py
879245009487
123456789abcdef
(.venv) napiea@raspberrypi:~/Documents/RFID $ |
```

Figure 5: RC522 Write and Read Operations

Lastly, I tested the ultrasonic ranger using the guide provided [4], with a sample output shown in *Figure 6*. I again modified this code slightly to have the display show the most recent reading. This sensor overall was one of my favorites based on how sensitive the response was to movement in front of the sensor. Distance measurements are derived from time-of-flight calculations based on the speed of sound.



```
napiea@web1: ~/public_html X
napiea@raspberrypi: ~/Docu X + v
distance is: 39.5 cm
|
```

Figure 6: Ultrasonic Ranger HC-SR04

Conclusion

Overall, this lab was straightforward to accomplish and provided a solid introduction to Linux-based workflows, SSH usage, and basic IoT sensor integration. While the individual tasks were well-documented, several practical challenges were encountered during setup.

One initial difficulty involved configuring the Linux virtual machine, particularly understanding package manager lock states during system updates, which temporarily delayed software installation. Additionally, early progress on the Raspberry Pi setup was hindered by the absence of a microSD card, highlighting the importance of verifying hardware requirements prior to beginning the lab. During sensor integration, the DHT11 sensor proved sensitive to wiring length and physical connections, requiring careful cable management to obtain stable readings.

From a security perspective, one area for improvement is the use of plaintext credentials in configuration files during Raspberry Pi setup. This approach contradicts best practices for IoT systems. A GUI-based setup process or the use of key-based authentication would better align the lab with secure system design principles.

Despite these challenges, the lab effectively reinforced foundational skills that will be essential for future coursework involving Linux-based simulation tools and networked IoT systems. These skills will be directly applicable in future labs involving GNURadio, NS-3, and networked IoT experimentation.

Acknowledgements

The author acknowledges the use of ChatGPT as a supplementary tool for report refinement, Linux command clarification, and troubleshooting guidance during the completion of this lab. All experimental work, system setup, code execution, and results were independently performed by the author.

References

[1] Q. Zhu, "DHT11 Temperature and Humidity Sensor,"

<https://qizhu8.github.io/PersonalWebsite/build/html/Course%20Content/IoT/Resources/Sensors/DHT11/DHT11.html>

[2] "Raspberry Pi GPIO Pinout,"

<https://pinout.xyz/>

[3] Q. Zhu, "RC-522 RFID Reader Module,"

<https://qizhu8.github.io/PersonalWebsite/build/html/Course%20Content/IoT/Resources/Sensors/RC-522/RC522.html>

[4] Q. Zhu, "HC-SR04 Ultrasonic Distance Sensor,"

<https://qizhu8.github.io/PersonalWebsite/build/html/Course%20Content/IoT/Resources/Sensors/HC-SR04/HCSR04.html>

Appendix

DHT11 CODE (Python script used to read temperature and humidity values)

```
1  #!/usr/bin/env python3
2
3  import RPi.GPIO as GPIO
4  from TEMP.DHT11_lib import DHT11
5  import time
6  import datetime
7  import sys
8
9  # initialize GPIO
10 GPIO.setwarnings(False)
11 GPIO.setmode(GPIO.BCM)
12 # read data using BCM pin 17 (physical pin 11)
13 instance = DHT11(pin=17)
14
15 # DHT11 hardware needs a 1 second sampling interval
16 MIN_SAMPLE_INTERVAL = 1.0
17 CLEAR_SCREEN = "\033[2J\033[H"
18 last_valid_status = "Last valid: waiting for sensor"
19 last_attempt_status = "Last attempt: not started"
20
21 try:
22     while True:
23         loop_started = time.monotonic()
24         result = instance.read()
25         now = datetime.datetime.now()
26
27         if result.is_valid():
28             temp_c = result.temperature
29             temp_f = temp_c * 9 / 5 + 32
30             last_valid_status = "Last valid: {%Y-%m-%d %H:%M:%S} | {:>3} C | {:>3} F | {:>2} %".format(
31                 now,
32                 temp_c,
33                 temp_f,
34                 result.humidity,
35             )
36             last_attempt_status = "Last attempt: success"
37         else:
38             last_attempt_status = "Last attempt: error {d} at {:%H:%M:%S}".format(
39                 result.error_code,
40                 now,
41             )
42
43     status = "{ }\n{ }".format(last_valid_status, last_attempt_status)
44     sys.stdout.write(CLEAR_SCREEN + status)
45     sys.stdout.flush()
46
47     sleep_for = max(0.0, MIN_SAMPLE_INTERVAL - (time.monotonic() - loop_started))
48     time.sleep(sleep_for)
49 except KeyboardInterrupt:
50     print()
51 finally:
52     GPIO.cleanup()
```

RFID READ (Python script used to read RFID)

```
RFID > read.py > ...
1  import RPi.GPIO as GPIO
2  from mfrc522 import SimpleMFRC522
3
4  reader = SimpleMFRC522()
5
6  try:
7      id, text = reader.read()
8      print(id)
9      print(text)
10 finally:
11     GPIO.cleanup()
```

RFID WRITE (Python script used to write to RFID)

```
RFID > write.py > ...
1  import RPi.GPIO as GPIO
2  from mfrc522 import SimpleMFRC522
3
4  reader = SimpleMFRC522()
5
6  try:
7      text = input('New data:')
8      print("Now place your tag to write")
9      reader.write(text)
10     print("Written")
11 finally:
12     GPIO.cleanup()
```

RANGER (Python script used to measure distance using ultrasonic time of flight sensor)

```
RANGER > ranger.py > ...
9  GPIO.setwarnings(False)
10 GPIO.setmode(GPIO.BCM)
11 GPIO.cleanup()
12
13 TRIG = 4
14 ECHO = 17
15
16 GPIO.setup(TRIG, GPIO.OUT)
17
18 instance = HCSR04(TRIG_pin=TRIG, ECHO_pin=ECHO) # BCM17
19
20 CLEAR_SCREEN = "\033[2J\033[H"
21 # keep a short pause between measurements to reduce sensor noise
22 REFRESH_DELAY = 0.1
23
24 instance.init_HCSR04()
25
26 try:
27     while True:
28         distance = instance.measure_distance()
29         sys.stdout.write(CLEAR_SCREEN + "distance is: {:.1f} cm\n".format(distance))
30         sys.stdout.flush()
31         time.sleep(REFRESH_DELAY)
32 except KeyboardInterrupt:
33     pass
34 finally:
35     GPIO.cleanup()
```